

#2. Preprocessor directives

You use a preprocessor directive every time you compile a program that includes a header file:

```
#include <stdio.h> ← This is a preprocessor directive.
```

The preprocessor scans through your C source file and generates a modified version that will be compiled. In the case of the `#include` directive, the preprocessing inserts the contents of the `stdio.h` file. Directives always appear at the start of a line, and they always begin with the hash (#) character. The next most common directive after `#include` is `#define`:

```
#define DAYS_OF_THE_WEEK 7
...
printf("There are %i days of the week\n", DAYS_OF_THE_WEEK);
```

The `#define` directive creates a *macro*. The preprocessor will scan through the C source and replace the macro name with the macro's value. Macros aren't variables because they can never change at runtime. Macros are replaced *before* the program even compiles. You can even create macros that work a little like functions:

```
#define ADD_ONE(x) ((x) + 1) ← x is a parameter to the macro.
...
printf("The answer is %i\n", ADD_ONE(3)); ← Be careful to use parentheses with macros.
                                     ← This will output "The answer is 4."
```

The preprocessor will replace `ADD_ONE(3)` with `((3) + 1)` before the program is compiled.

Conditions

You can also use the preprocessor for **conditional compilation**. You can make it switch parts of the source code on or off:

```
#ifdef SPANISH ← If the SPANISH macro exists...
char *greeting = "Hola"; ← ...include this code.
#else
char *greeting = "Hello"; ← If not, include this code.
#endif
```

This code will be compiled differently if there is (or isn't) a macro called `SPANISH` defined.